

FINAL YEAR PROJECT · DEFENSE SCRIPT

ForgeCloud

A miniature, self-hosted deployment platform

Presenters: Speaker A · Speaker B

Format: Two-person spoken script, ~18 minutes + Q&A

Built with: TypeScript · Node.js · Fastify · BullMQ · PostgreSQL · Redis · Docker · Next.js

Scale: 10 workspaces · 159 source files · ~27,400 lines · 11 of 13 phases complete

How to use this document. Everything in **bold-ish plain text** is meant to be said out loud — but say it in your own words, don't read it. Lines marked *[cue]* in grey are stage directions, not speech. Part 8 is the live-demo narration; Part 9 is the question bank — read Part 9 twice, it's where defenses are actually won or lost. Replace "Speaker A" and "Speaker B" with your names before printing.

Part 0 · Before you walk in — the 60-second version

If the panel gives you one minute and nothing else, this is what you say. Everything after this page is elaboration.

THE ELEVATOR PITCH

ForgeCloud is a small deployment platform — the kind of thing Vercel, Netlify or Heroku are at industrial scale. You create a project, upload your app, and press Deploy. From there the system takes over: it puts the deployment on a queue, a separate worker process picks it up, unpacks the code into an isolated sandbox, runs the install and build commands, packages the result into a Docker image, starts a container with strict resource limits, health-checks it over HTTP, and marks it LIVE — while every log line and every state change streams into your browser in real time over a WebSocket.

If the build fails, it retries with backoff. If the retries run out, it goes to a dead-letter queue and the deployment is marked failed with the real reason. If the worker machine dies mid-build, another worker notices and takes the job over. If the new version is bad, you can roll back to the last good one. And all of it runs on one laptop, but it is designed so that every piece could be moved onto its own machine without changing the shape of the code.

The three sentences that matter most

1. **What it is:** a working miniature cloud platform that takes source code and turns it into a running, monitored, health-checked container — with a live web dashboard.
2. **Why it's hard:** it is a distributed system. Multiple processes have to agree on state, logs have to stream while they're being produced, failures have to be survivable, and the platform executes code it did not write, which is a security problem.
3. **What we learned:** the actual internals — queues, worker processes, child processes, streams and backpressure, WebSocket fan-out, container orchestration, graceful shutdown, and keeping shared state outside process memory.

Things to never say

- "It's basically a Vercel clone." It isn't, and the panel will chase you into a comparison you will lose. Say "it's a working miniature of the same architecture, built to learn the internals."
- "We used a library for that." Not for the parts that matter — see Part 5. Know which pieces are hand-written and be proud of them.
- "It works" — without saying *how you know*. Every claim in this script maps to something recorded in the project's build log with the command that proved it.

SPEAKER A Good morning. I'm Speaker A, this is Speaker B, and our project is called **ForgeCloud**. It is a self-hosted deployment platform — a small, working version of the kind of system that sits behind Vercel, Netlify, Railway or Heroku.

The one-line description is this: **you give it source code, and it gives you back a running, health-checked, monitored application — and it shows you every step of that happening, live.**

SPEAKER B To explain why we built it, it helps to say what deploying an application actually involves, because it is a lot more than copying files to a server.

You have to get the source code somewhere. You have to install its dependencies. You have to run its build. You have to package the result so it runs the same way every time. You have to start it in an isolated environment with limits on how much memory and CPU it can take. You have to check that it actually came up healthy before you send users to it. And if any of those steps fails, you have to fail *safely* — not leave a half-started application serving errors.

SPEAKER A Commercial platforms hide all of that behind one button. That's a good product decision and a terrible learning experience — because the interesting engineering is precisely the part they hide.

So we built the hidden part. ForgeCloud is that pipeline, visible, running on one laptop, with every stage exposed on a dashboard.

SPEAKER B And we want to be honest about the scope, because it shapes every decision we made. **This is a learning project, not a production platform.** Where a shortcut would have made the code simpler but taught us nothing, we deliberately did not take it. Where hand-rolling something would have been risky and taught us little, we used a well-established library and we can tell you exactly why. That trade-off is the theme of this whole presentation.

IF ASKED EARLY: "WHY NOT JUST USE DOCKER COMPOSE / A SHELL SCRIPT?"

Because a shell script has no memory. It can't tell you a deployment is on attempt two, it can't stream its output to five browsers at once, it can't hand a half-finished job to another machine when the first one dies, and it can't refuse to start a second container for a project that already has one running. Every one of those is a distributed-systems problem, and every one of them is in our system.

SPEAKER A Before the architecture, let me walk through it from the user's side, so the rest of the talk has something concrete to attach to.

You register an account. That creates you as a user and puts you into an **organisation** — a team. Inside an organisation you create **projects**. A project is the unit you deploy: it carries a name, the command to install dependencies, the command to build, the command to start the app, which port the app listens on, and the URL path we should hit to decide whether it's healthy.

SPEAKER B A project also carries **environment variables** — configuration and secrets. Anything marked as a secret is encrypted before it touches the database, and it is never returned in plaintext by the API and never written to a log. We'll come back to that in the security section, because it's one of the rules we set at the start and enforced in code rather than by remembering to be careful.

SPEAKER A Then you upload your application as an archive, and you press **Deploy**.

From that moment you don't touch anything. The dashboard shows a pipeline that advances by itself — queued, assigned to a worker, cloning, installing, building, creating container, starting, health check, live. Underneath it, the actual output of `npm install` and `npm run build` scrolls past in real time. Not a summary afterwards — the real output, line by line, as the process produces it.

SPEAKER B When it reaches **live**, the dashboard gives you a URL. You click it and your application is there, running in a container on the machine.

And from that point the platform keeps watching: CPU and memory for that container, the health of the system, the depth of the job queue, how long each deployment took. If you deploy a bad version, there's a rollback button that brings the previous good one back.

What the dashboard shows

Ten screens, all driven by the same live data:

SCREEN	WHAT IT DOES
Login / Register	Account creation and sign-in
Organisation home	Project list for the current team
Project detail	Settings, env vars, source upload, deploy button, build history, and the live pipeline + log stream
Members	Team members and their roles — the RBAC view
API keys	Create, list and revoke programmatic access keys
Containers	Every running container, its URL, and its live CPU / memory
Metrics	Charts: event-loop lag, CPU, memory, queue depth, deployment duration
Fleet	The load balancer's request split, each API replica, each worker, and the dead-letter queue

SPEAKER B Now the shape of the system. There are **four separate programs** and **two shared data stores**, and the separation between them is the single most important design decision we made.

The four programs

COMPONENT	RESPONSIBILITY	WHAT IT DELIBERATELY CANNOT DO
Dashboard Next.js	The web interface. Talks to the API over HTTP for actions and over a WebSocket for live updates.	Holds no authority — it only displays what the server tells it.
API Fastify	Authentication, permissions, all CRUD, creating deployments, and the WebSocket gateway. 52 HTTP endpoints.	Never runs a build. It doesn't import Docker and it doesn't spawn processes. It only writes a row and puts a job on a queue.
Worker Node process	Takes jobs off the queue and executes the whole deployment pipeline: unpack, install, build, containerise, start, health-check.	Never serves HTTP. It has no routes and nothing can call into it directly — the only way to give it work is through the queue.
Proxy hand-written	A reverse proxy / load balancer sitting in front of several API copies, so the browser talks to one address.	Holds no database connection, no Redis connection, and no credentials — deliberately, so it can be killed freely.

SPEAKER A The reason the API and the worker are separate programs and not two functions in one program is the whole point. A build is slow, unpredictable, and it runs code we did not write. If we ran it inside the web server, one bad build would block every other request on that process — and one crash would take the website down with it.

By splitting them, the API stays fast and responsive no matter what a build is doing, and we can run *more workers* when there's more work without touching the API at all. That's what horizontal scaling actually means, and it's the reason platforms are built this way.

The two shared stores

SPEAKER B **PostgreSQL is the source of truth.** Everything durable lives there — users, teams, projects, deployments, every state change, worker registrations, API keys, file records, audit logs. If every process on the machine died right now, the database would still be able to tell you exactly what state every deployment was in.

Redis is the coordination layer. It holds the job queue, the publish/subscribe channels that carry logs and status updates between processes, login sessions, rate-limit counters, distributed locks, and worker heartbeats. It's the fast, temporary, shared scratchpad.

THE GOLDEN RULE — SAY THIS ONE CLEARLY, IT'S THE STRONGEST IDEA IN THE PROJECT

No authoritative state ever lives in a single process's memory.

If any one process is holding the only copy of something important, then that process cannot be killed, cannot be restarted, and cannot be duplicated — and the whole system becomes one machine pretending to be a platform. Because we pushed all shared state into Postgres and Redis, we can run two API copies and three workers, kill any of them mid-deployment, and the system carries on. That's not an accident, and it's exactly what we demonstrate live.

How a deploy travels through the system

```
Browser → Proxy → API → writes deployment row to Postgres
                        → pushes job onto the Redis queue → responds 202 Accepted

Worker → pulls the job → runs the pipeline → writes each state change to Postgres
                                           → publishes each log line + status to
Redis

Redis → every API copy is subscribed → pushes down the WebSocket → Browser
```

SPEAKER A Notice what that gives us for free. The worker doesn't know or care which browser is watching, or how many, or which API process they're connected to. It publishes to a Redis channel and walks away. Every API copy is subscribed to that channel, so *whichever* API copy your browser happens to be attached to will receive the message and forward it to you.

That is how two people, on two different browsers, connected to two different API processes, watch the same deployment advance in perfect sync. We tested exactly that — both tabs received the identical nine-status sequence and all 146 log lines.

SPEAKER B Let me take you through what the worker actually does, stage by stage. A deployment has **thirteen possible states** — nine on the happy path, four terminal.

queued → assigned → cloning → installing → building → creating_container → starting → health_check → **live**

terminal: failed · stopped · rolled_back · canceled

STAGE	WHAT ACTUALLY HAPPENS
queued	The API has written the row and put a job on the queue. Nothing is running yet. If two workers are idle, they compete for it — the queue guarantees exactly one wins.
assigned	A worker has claimed the job and stamped its own ID on the deployment row. There is a guard here: a worker will refuse to claim a deployment that another <i>live</i> worker already owns.
cloning	The source archive is fetched from storage and unpacked into a fresh sandbox directory, one per attempt. Every path inside the archive is checked — an entry trying to escape the sandbox is rejected, not extracted.
installing	The project's install command runs as a real child process. Its output is captured as a stream and pushed out live.
building	Same again for the build command. Both stages have timeouts; if one hangs, the process is killed and the deployment fails with a real reason rather than hanging forever.
creating_container	The built directory is packaged, a Dockerfile is generated for it, and Docker builds an image. The image is the deployable artifact — tagged with the project and the deployment ID so it can be reused for a rollback.
starting	A container is created from that image with hard limits applied, environment variables injected at creation time, and started. Docker assigns a host port and we record it.
health_check	We repeatedly make an HTTP request to the app's health path until it answers successfully or the timeout expires. This is the gate. A container that starts but never answers is not a successful deployment.
live	Status, URL, container ID, host port and total duration are recorded, the project's "active deployment" pointer is swapped to this one, and a LIVE event is published.

THE RULE THAT MAKES THE DASHBOARD POSSIBLE

Every single one of those transitions does two things, always, without exception: **it is written to Postgres** — both as a row in an append-only event log and as a status column on the deployment for fast reads — **and it is published to Redis** on that deployment's channel.

The append-only event log is why you can open a deployment that finished an hour ago and still replay its entire history in order. The published message is why you see it happen live. Same transition, two consumers, no extra code at the call site.

SPEAKER A And every stage cleans up after itself on failure. If the build fails, the sandbox is removed. If the container starts but fails its health check, the container is stopped *and removed* — we don't leave dead containers accumulating on the machine. There's also a sweeper that catches sandboxes left behind by a worker process that died before its cleanup could run.

SPEAKER B This is the section that answers "what did you actually build, versus what did a library build for you?" — and we'd rather raise it ourselves than have it raised for us.

We set a rule at the start. **Where the mechanism was the thing we wanted to learn, we wrote it ourselves. Where the mechanism was well-understood but easy to get catastrophically wrong, we used a proven library and we can defend the choice.**

Written by hand — seven of them

1. The realtime layer

We use the raw WebSocket library, not a framework like Socket.IO. That means we own the connection lifecycle, the message format, the subscription registry — which socket is listening to which topic — the heartbeat that detects dead connections, and the bridge from Redis publish/subscribe out to the right sockets. There is a versioned message contract, a cap on how many topics one socket may subscribe to, a maximum inbound frame size, and a send-buffer limit so one slow browser can't grow our memory without bound.

2. Build execution via child processes

Install and build run through Node's `spawn`, driven by us — not by a helper that returns a string when it's done. We handle the output streams, the exit code, the timeout, and killing the process when it overruns. Critically, we pass the command as an *array of arguments with the shell disabled*, which is what makes command injection structurally impossible rather than merely unlikely.

3. Log streaming

This one is worth dwelling on. The naive approach is to collect all the output into a string and save it when the build finishes. We don't do that, for two reasons: you'd see nothing until the end, and a build that prints a gigabyte would take the process down with it.

Instead the output is a genuine stream. It passes through a transform we wrote that splits the byte chunks into complete lines — and that's a real problem, because a chunk from the operating system can end in the middle of a line, or even in the middle of a multi-byte character. Each complete line is then published to Redis, written to a log file, and recorded in the event log, as it happens. There are caps on total log size and total number of log events so a runaway build is truncated rather than fatal.

4. Container orchestration

We talk to the Docker Engine API directly rather than shelling out to the `docker` command. We create the image, create the container with its limits, start it, read back the port Docker assigned, and stream its logs. Docker's log stream is multiplexed — stdout and stderr are interleaved in one stream with an eight-byte header on each frame telling you which is which and how long it is. **We demultiplex that by hand** rather than using the library's convenience helper, because that framing is exactly the kind of thing worth understanding once.

5. Graceful shutdown

Every long-running process handles termination signals properly. On shutdown the API stops accepting new connections, lets in-flight requests finish, then closes the database pool and Redis — with a hard timer so it can't hang forever. The worker stops taking new jobs, aborts the build it is running, records that build as failed with a real reason so the row isn't left lying, and exits. The difference between a clean shutdown and a kill is visible on the dashboard, and we demonstrate both.

6. Observability

We measure ourselves rather than importing an agent. Event-loop lag comes from Node's high-resolution monitor — that's the number that tells you whether the process is actually responsive, not just alive. CPU comes from process CPU accounting, memory from process memory usage, container statistics from Docker's own API, and queue depth from the queue itself. It's exposed in Prometheus's text format at a metrics endpoint and also streamed to the dashboard over the same WebSocket as everything else.

7. The reverse proxy

The load balancer in front of the API copies is about a thousand lines of Node that we wrote. Nginx would do this in ten lines of configuration — which is exactly the reason we wrote it. The interesting parts are invisible in a config file: which HTTP headers must not be forwarded, when it is *safe* to retry a failed request, and how to relay a WebSocket upgrade, which means replaying the handshake and then joining two raw sockets together.

Chosen deliberately, not written by hand

LIBRARY	THE HONEST REASON
BullMQ (queue)	Retries, exponential backoff, delayed jobs, dead-letter handling and job locking. Every one of those is correctness-critical and subtly hard. Getting the locking wrong means two workers building the same deployment — a real bug with real consequences and very little learning payoff.
Kysely (database)	It's a typed query builder, not an ORM. We write actual SQL and get compile-time types back. It hides nothing about what query runs, which is the opposite of what a heavyweight ORM does.
Fastify (web server)	Plugin encapsulation, schema-based validation, and structured logging built in. Writing our own HTTP router would have taught us routing, which is not one of our learning goals.
argon2 (passwords)	Never hand-roll password hashing. Ever. That's not a learning goal, that's a vulnerability.

SPEAKER A Anything can deploy an app when everything works. The reason this is an engineering project rather than a coding exercise is what happens when things don't.

Retries with backoff

SPEAKER B When a deployment fails for a reason that might be temporary — the health check timed out, the container wouldn't start — the queue retries it with **exponential backoff**: five seconds, then ten, then twenty. Backing off matters. Retrying instantly hammers a system that is already struggling; each attempt is numbered and recorded, so the dashboard shows you "attempt 2 of 3."

Importantly, we distinguish retryable failures from ones that aren't. If your code has a syntax error, the build will fail identically three times in a row. Those failures skip the retries and go straight to the dead-letter queue, because retrying a certainty is just wasted time.

The dead-letter queue

When the attempts run out, the job doesn't vanish and it doesn't get retried forever. It moves to a separate **dead-letter queue** — a holding area for jobs that failed permanently — and the deployment is marked failed with the captured reason. The dead-letter queue is visible on the Fleet page. That's what "failing safely" means: nothing is lost, and a human can see exactly what died and why.

Idempotency — the double-click problem

SPEAKER A Here's a question worth asking any deployment system: what happens if the user double-clicks Deploy? Or their connection drops after the request was sent but before the response arrived, and the browser retries?

Naively, you get two deployments and potentially two containers of the same app fighting over the same port. We handle it two ways. First, deployments are unique on the combination of project, source version and an **idempotency key** sent by the client — so a duplicate request returns the *existing* deployment instead of creating a new one. Second, there is a **distributed lock in Redis** that enforces one starting-or-live container per project. Even if two requests somehow got through, only one can hold that lock.

Rollback

Because the image built for each deployment is kept and tagged, rolling back isn't "undo" — it's a fresh deployment started from a previous successful deployment's image. It goes through the same pipeline and the same health check, and only when it passes do we swap the active pointer and stop the old container. It's recorded as a deployment with a link back to the one it restored. That means **a rollback can itself fail safely**, which an in-place revert cannot.

THE DEMO THAT BEST PROVES THE ARCHITECTURE — WORKER CRASH RECOVERY

Set-up: two workers running. Start a deployment. Wait until it's mid-build. Then **SIGKILL** the worker that's building it — not a polite shutdown, an instant kill with no chance to clean up.

What happens: nothing is recorded, because the process had no chance to record anything. The deployment row is left saying "building" and naming a process that no longer exists. Then three independent mechanisms converge:

① The dead worker's heartbeat key in Redis stops being refreshed and expires on its own. ② The queue stops seeing that job's lock renewed and returns the job to the queue. ③ The next worker to pick it up proves the previous owner is really gone, moves the row back to "queued" — *recorded as a real state transition, with a message naming the dead worker* — claims it, and runs it again.

Measured recovery: 30 to 32 seconds, twice, and the deployment finished live. The timeline on the dashboard literally reads: *"Worker worker-2 stopped responding during 'building'; the job was returned to the queue and picked up by worker-1."*

And there's a backstop underneath that: if the queue *also* lost the job, a periodic sweep — run by one elected process, not all of them — finds the abandoned deployment and marks it failed with "worker lost", rather than leaving a spinner on someone's dashboard forever.

Graceful shutdown, as contrast

The same worker, given a polite **SIGTERM** instead, behaves completely differently: it stops accepting new jobs, aborts its current build, records it as failed with a "build aborted" reason, and the retry lands on the other worker **within about six seconds** — because nothing had to time out. The contrast between 30 seconds and 6 seconds is the entire value of handling shutdown signals properly, and it's why we demo both.

**SPEAKER
B**

We should be explicit about something: **this platform executes code it did not write**. That is not a footnote, it's the central security fact of the project. Somebody uploads an archive and we run its install script. If we're careless, that's a remote code execution vulnerability with a web form attached to it.

The rules, and how each is enforced

THREAT	ENFORCEMENT
Command injection	Commands are never passed to a shell. We spawn with an explicit argument array and the shell disabled, so a project whose build command contains <code>; rm -rf /</code> gets those characters passed through as literal text to a program that doesn't exist — it fails, it doesn't execute.
Path traversal	Every filesystem path the pipeline touches — including every entry inside an uploaded archive — is resolved and checked against the sandbox root. Anything escaping it is rejected before extraction.
Resource exhaustion	Containers run with a hard memory cap, swap disabled (so exceeding memory kills the container instead of thrashing the host), a CPU quota, and a process-count limit that stops a fork bomb.
Container escape / privilege	All Linux capabilities dropped, no-new-privileges set, read-only root filesystem where possible, non-root user inside the container, its own network rather than the host's.
Runaway output	Caps on log bytes, log events, extracted archive size and extracted file count. Timeouts on every child process and every external call.
Secrets at rest	Env var values are encrypted with AES-256-GCM — authenticated encryption, so tampering is detected, not just decryption failing quietly. The stored format carries a version byte specifically so keys or algorithms can be rotated later without orphaning old rows.
Secrets in logs	Enforced by the logger itself, which has a redaction list — passwords, tokens, secrets, auth headers, cookies, encrypted values. It is not enforced by remembering to be careful, because that always fails eventually.
Passwords	argon2id at the OWASP-recommended baseline. Sessions are opaque random tokens, and Redis stores only a hash of the token — so a dump of Redis cannot be replayed as a login.
Authorisation	Four roles — owner, admin, member, viewer. Every mutating endpoint requires authentication, a role check, schema validation of its input, and passes a rate limiter.

WORTH SAYING OUT LOUD — THE HONEST BOUNDARY

There are two different protections doing two different jobs. The **sandbox** provides containment during the *build* — our own paths cannot be escaped. The **container** provides isolation at *run time*. But the install and build commands themselves still run as the worker's own user on the host machine, not inside a container. We know that, it's written down in our documentation as a known limitation, and building inside a container is the obvious next hardening step. **We'd rather tell you that than have you find it.**

Proving it scales horizontally

SPEAKER A Everything runs on one laptop — but we can prove the architecture is genuinely horizontal, not just claimed to be. One command starts a cluster: several API copies behind the proxy, several workers competing for the same queue.

Two things make this work, and both come back to the golden rule. Sessions live in Redis rather than in an API process's memory, so **there are no sticky sessions** — any API copy can serve any request from any user. And the queue guarantees a job goes to exactly one worker, so adding workers adds parallel builds with no coordination code from us.

We verified it: six requests round-robbined across two replicas; the same user's session worked on both; killing a replica mid-request meant it was dropped from rotation and **the request was retried on the survivor so the browser saw a 200, not an error**; and a WebSocket relayed through the proxy carried 245 metric frames in eight seconds.

SPEAKER B And that's the closing architectural point. To take this from one laptop to real infrastructure, you don't restructure anything. You point at a managed Postgres and a managed Redis instead of local ones, you put each program on its own machine, and you point the workers at a real Docker host or an orchestrator. **The change is configuration, not code shape.** That's the difference between a demo and an architecture.

Part 8 · Live demo narration

≈ 5 min

One person drives, one person narrates. Do not swap mid-demo. Have the cluster already started and logged in before you begin — never start infrastructure in front of a panel.

[cue] Before the panel arrives: cluster running with 2 API replicas + 2 workers, browser logged in, project created, sample app already uploaded, and a second browser tab open on the same deployment page. Have a terminal visible for the kill command.

Beat 1 — The happy path

~90 seconds

NARRATOR "This is a project. It knows how to install itself, how to build itself, which port it listens on, and where to check its health. I press Deploy."

"The button returned immediately — the API did not build anything, it wrote a row and queued a job. That's why it's instant."

"Now watch the pipeline. Queued. Assigned — a worker has claimed it, and it's named there. And now the log panel: that's the real output of npm install, streaming as the process produces it, not a recording played back afterwards."

"Building. Creating container — that's Docker building an image. Starting. Health check — we're making HTTP requests to the app until it answers. And... live."

[cue] Click the URL. The app opens. Let it sit on screen for two full seconds — this is the payoff shot.

Beat 2 — Two tabs, one truth

~30 seconds

Show the second tab, which followed the identical sequence. Narrate: *"These two tabs are connected to two different API processes. Neither of them is talking to the worker. The worker published to Redis and every API copy forwarded it. That's the fan-out."*

Beat 3 — Metrics

~30 seconds

Open the Metrics page while the deployment is still fresh. Point at event-loop lag specifically: *"That number is how responsive the process actually is. A process can be alive and still be useless — this is the one that tells you which."* Then the container's CPU and memory, and queue depth.

Beat 4 — Failure and retry

~60 seconds

Deploy something that fails — a broken build command is the cleanest. Narrate the failure being *captured*, not *swallowed*: the real error, the exit code, the attempt counter, and the deployment marked failed rather than stuck.

Beat 5 — Rollback

~45 seconds

Press rollback. Narrate: *"This isn't an undo. It's a new deployment from the previous good image, and it goes through the same health check. Only when it passes do we swap over and stop the old container — so the rollback itself can fail safely."*

NARRATOR "Last one. I'm starting a deployment, and while it's building I'm going to kill the worker doing the work. Not shut it down — kill it. SIGKILL. It gets no chance to clean up, no chance to record anything."

[cue] Type `kill worker-2` in the cluster terminal, mid-build. Then go back to the dashboard and let it sit. This takes ~30 seconds — do not fill the silence with apologies. Use it.

NARRATOR "Right now the database says that deployment is building, on a process that no longer exists. Nothing knows it's dead yet. Three things are about to happen: the dead worker's heartbeat expires, the queue notices the job's lock isn't being renewed and puts it back, and the surviving worker proves the old owner is gone before taking it."

[when it moves] "There. It went back to queued — and read the message, it names the worker that died and the stage it died in. Now worker-1 has it, attempt two, and it's running the pipeline from the start. Because the sandbox died with the old worker, so a partial install can't be trusted — it rebuilds, which is slower but correct."

[when live] "And live. Nothing was lost. Nobody intervened. That works because no authoritative state was ever in that worker's memory."

DEMO FAILURE PROTOCOL — AGREE ON THIS BEFOREHAND

If something breaks live: **do not apologise repeatedly and do not start debugging**. Say one sentence — *"That's not the behaviour we recorded; let me show you the verified run"* — and move to the build log, which has every one of these results recorded with the exact commands that produced them. Recovering calmly reads as competence. Panicking reads as not knowing your own system.

Decide right now which of you says that sentence, so you don't both talk at once.

Part 9 · Question bank

Read this section twice. Most defense marks move here, not during the presentation.

On scope and originality

Isn't this just a worse version of Vercel?

Yes, in the sense that a model bridge is a worse version of a real bridge. It's not competing with Vercel — the goal was to build the mechanism they hide, and we did: the queue, the worker pool, the streaming, the container lifecycle, the failure recovery. What we don't have is scale, multi-tenancy across machines, a CDN, and years of hardening. What we do have is every architectural piece, and we can explain each one.

What is genuinely yours versus the libraries?

Seven subsystems are hand-written: the WebSocket layer and its fan-out, build execution through child processes, the log streaming pipeline, the container orchestration including demultiplexing Docker's log framing by hand, graceful shutdown, the metrics collection, and the reverse proxy. We used four significant libraries and each was a deliberate call: BullMQ because getting job locking wrong means two workers building the same thing; Kysely because it's a typed query builder rather than an ORM, so we still write SQL; Fastify because HTTP routing isn't a learning goal; and argon2 because nobody should ever hand-roll password hashing.

How big is it, really?

Ten workspace packages, 159 hand-written TypeScript source files, roughly 27,400 lines, 52 HTTP endpoints, 7 database migrations, 10 dashboard screens. TypeScript in strict mode throughout, and the type checker passes clean across every workspace.

On the architecture

Why separate the API and the worker? Why not one program?

Three reasons. Isolation: a build is slow and runs untrusted code, so running it inside the web server means one bad build blocks every other request and one crash takes the site down. Scaling: they scale on different axes — you might need more workers without needing more API capacity. And resilience: because they're separate and communicate through a queue, killing a worker mid-build is survivable. Fusing them would break all three.

Why both Postgres and Redis? Isn't one enough?

They do different jobs. Postgres is durable and transactional — it's the source of truth, and if everything crashed it would still know the exact state of every deployment. Redis is fast and ephemeral, and it gives us three things Postgres doesn't do well: publish/subscribe for the live fan-out, queue primitives with atomic locking, and cheap expiring keys for sessions, rate limits and heartbeats. Doing pub/sub in Postgres is possible but awkward; doing durable state in Redis is asking to lose it.

What happens if Redis goes down?

Deployments stop — you can't queue what has no queue — but the system degrades visibly rather than crashing silently. The health endpoint reports which dependency is down and why, and returns a 503, so it works as a real readiness probe. Queue commands have a deadline so a dead Redis fails a request in seconds instead of hanging it. Honest answer: making that path fully graceful is part of our final phase, which isn't complete.

How do you stop two workers building the same deployment?

Three layers. The queue itself guarantees a job is delivered to exactly one consumer. The claim step in the database refuses a deployment already owned by a worker whose heartbeat is still alive. And a Redis lock enforces one starting-or-live container per project. The interesting case is the crashed worker — we can't just take the row, because the previous owner might be alive and slow rather than dead. So the takeover requires proving the heartbeat key has actually expired.

Why is the proxy hand-written instead of Nginx?

Precisely because Nginx would have been ten lines of config. The parts a config file hides are the parts worth learning: which headers must not be forwarded, when retrying a failed request is safe — only before the body has started streaming, otherwise you'd send it twice — and how a WebSocket upgrade is actually relayed, which means replaying the handshake and then piping two raw sockets together. It also deliberately holds no state and no credentials, so it's the one process we can kill freely.

On the technical detail

What is backpressure and where does it appear here?

Backpressure is a fast producer being told to slow down by a slow consumer. It appears in three places for us. Build output: a build can print faster than we can publish, so it's a stream with flow control rather than a buffer we append to. File upload and download: streamed, so a 50 MB upload doesn't become 50 MB of memory. And the WebSocket: if a browser reads slowly, we cap the send buffer and drop rather than growing memory without bound — because unbounded buffering isn't handling backpressure, it's postponing a crash.

Why line-split the output? Why not just forward the chunks?

Because a chunk from the operating system has no relationship to a line. It can end mid-line or even mid-character in UTF-8. Forwarding raw chunks would give the browser fragments to reassemble and would corrupt any non-ASCII output. So a transform stream buffers the partial tail and only emits complete lines — which is also what makes a log line a meaningful unit we can count, cap and store.

Why use worker threads at all — isn't Node single-threaded?

Node's event loop is single-threaded, which is exactly the problem. Compressing a build artifact and hashing it are CPU-bound — do them on the main thread and the event loop stops, so heartbeats stop, the health check stops answering, and the process looks dead while it's just busy. Moving them to a worker thread keeps the loop free. It's also the clean illustration of threads versus processes: threads for CPU work in-process, separate processes for isolation, which is what workers and containers are for.

Why is the health check a separate stage? The container started.

Because "the process started" and "the application works" are different claims. A container starts successfully and then crashes on its first database connection, or listens on the wrong port, or takes forty seconds to warm up. If we marked it live on start, we'd hand users a URL that returns errors. The health check is the gate — we only claim live once the app has actually answered a successful HTTP request. It's also the most common real-world deployment failure and it's why rollback can be safe.

Why keep both an event log and a status column? That's duplication.

It's deliberate denormalisation. The append-only event log is the full history — it's what lets us replay a deployment's whole story in order, and it's an audit trail. The status column is a denormalised cache of the latest state, so listing fifty deployments is one simple query instead of fifty aggregations. Both are written in the same transition, so they can't drift.

Why encrypt env vars rather than hash them?

Because we need them back. Hashing is one-way and correct for passwords, where you only ever need to verify. The worker must inject the actual values into the container at start-up, so it has to be reversible encryption. We use AES-256-GCM — authenticated, so tampering is detected rather than silently decrypting to garbage — and the stored format has a version byte at the front specifically so a key rotation later can still read old rows.

On limitations — answer these confidently, not defensively

What doesn't work? What's missing?

Four things, and they're all written down in our build log rather than discovered by you.

Automated tests. This is the biggest gap and we won't dress it up. Everything is verified — every claim we've made was proven by a run with commands and output recorded — but by scripts written during verification and then thrown away, not by a test suite that lives in the repository. The pure functions are the obvious first suite.

Builds run on the host. The sandbox stops path escapes, and the container isolates the app at run time, but the install and build commands themselves run as the worker's user. Building inside a container is the next hardening step.

The final two phases aren't done. Eleven of thirteen are complete. Phase 11 is the deliberate failure demonstrations and a formal security pass; Phase 12 is presentation polish and optional stretch goals like local TLS and custom domain routing. Interestingly, two of Phase 11's items got done early as a side effect of the scaling work.

Crash recovery restarts from the beginning. A taken-over deployment rebuilds from scratch rather than resuming mid-stage, because the dead worker's sandbox is gone and its partial dependency install can't be trusted. Correct, but not fast.

Recovery takes 30 seconds. Isn't that slow?

It's tunable, and 30 seconds is a chosen default rather than a limit. It's the job lock duration plus the scan interval. You could make it much shorter, but then a worker whose event loop is briefly blocked gets declared dead while it's still working — and now you have two workers building the same deployment, which is a far worse failure than waiting. We picked a value that's watchable in a demo and safe; the knobs are exposed in configuration.

You said a killed replica loses nothing. Is that exactly true?

Not exactly, and we should be precise. A request that fails to *connect* is retried on another replica and the browser sees no error — that's the case we demonstrate. But a request whose body had already started streaming, or whose response had already started, gets an error or a truncated response. Retrying those would require buffering every request in the proxy, which would undo the reason the proxy holds no state. That's the honest limit.

How would you take this to production?

In order: automated tests first, because everything else depends on being able to change code safely. Then run builds inside containers rather than on the host. Then replace local file storage with real object storage — it's already behind an interface shaped like S3 for exactly that reason. Then managed Postgres and Redis, each program on its own host, and workers pointed at a real orchestrator. And TLS, real domain routing, and per-organisation quotas. The architecture doesn't change — that's the point of the separation.

Which part was hardest?

Have an answer ready — a specific one. Good candidates: the crash-recovery path, because it required accepting that "went back to the queue because its worker vanished" is a legitimate state transition to model and record rather than an error to hide; the log streaming, because the naive buffer-it-all approach is so much easier and so much worse; or the WebSocket fan-out, because it's the piece that forced the golden rule about never holding state in one process. Pick one each and be able to go deeper on it.

Who did what?

Agree on this before you walk in and give the same answer. The cleanest split follows the architecture: one of you owns the platform side — API, auth, queue, database, proxy — and the other owns the execution side — worker, builds, Docker, streaming, metrics — with the dashboard and the design decisions shared. Both of you must be able to answer questions about both halves; a panel will test that.

Closing lines

SPEAKER A "To close: ForgeCloud takes source code and turns it into a running, health-checked, monitored container — and shows you every step as it happens. It survives a worker being killed mid-build, it retries with backoff, it fails into a dead-letter queue rather than silently, and it rolls back safely."

SPEAKER B "It runs on one laptop, but nothing about it assumes that. Every process is separate, all shared state is in Postgres and Redis, and moving to real infrastructure is a configuration change rather than a rewrite. What we set out to learn was how a deployment platform actually works underneath the button — and the honest answer is that it's a distributed system, and we now know why. Thank you — we're happy to take questions."